

얼굴 인식 스마트 도어락

본 보고서를 2025년도 Project [얼굴 인식 스마트 도어락]의
최종보고서로 제출합니다.

학부(전공)/학과: 컴퓨터공학과

담당(지도)교수: 최병윤

팀명: 7조

제출일: 2025년 12월 22일

팀 장 : 허진혁 (20212893)

팀원1 : 권태정 (20212909)

팀원2 : 이준영 (20212943)

팀원3 : 한경미 (20232910)

〈 요약 문 〉

프로젝트 목표 (300자내외)	본 프로젝트는 사물인터넷(IoT, Internet of Things)과 인공지능(AI, Artificial Intelligence) 기술을 융합한 임베디드 AIoT 시스템 구축을 목표로 한다. 기존의 물리적 열쇠나 비밀번호 입력 방식의 번거로움을 해소하고 보안성을 강화하기 위해, Raspberry Pi 기반의 독립형 안면인식 도어락 시스템을 설계하였다. 특히, 사용자가 도어락의 '열림' 버튼을 누르는 즉시 카메라 모듈이 활성화되어 실시간으로 객체 탐지 및 안면 인식을 수행하며, 사전 등록된 사용자로 판별될 경우 지체 없이 잠금 장치를 해제하는 반응형 보안 시스템을 구현하는 데에 주안점을 둔다.		
내용 (500자내외)	본 시스템은 Raspberry Pi 5와 Flask 웹 서버를 중추로 하여, 하드웨어 제어와 소프트웨어 알고리즘이 유기적으로 연동되도록 설계되었다. 주요 기능의 동작 원리는 다음과 같다. 1. 딥러닝 기반 안면 인식 및 제어 (AI Access Control) 도어락의 '열림' 신호가 인가되면 카메라는 전방의 객체를 촬영하고, YOLOv8 모델이 이를 실시간으로 추론한다. 단순한 얼굴 탐지를 넘어 얼굴이 카메라와 일정 거리 이내에 위치했는지 판별하는 근접도 필터링(Face Area Ratio) 알고리즘이 적용되어 오작동을 방지한다. 인증에 성공하면 GPIO를 통해 스텝 모터를 회전시켜 물리적 잠금을 해제하며, 동시에 부저(Buzzer)와 LED를 통해 사용자에게 직관적인 피드백을 제공한다. 2. 데이터셋 구축 및 관리 (Data Acquisition) 모델의 지속적인 성능 향상을 위해 웹 인터페이스 내에 이미지 캡처 기능을 내장하였다. 관리자는 /video_feed 엔드포인트를 통해 전송되는 실시간 영상을 모니터링하며, 버튼 클릭 한 번으로 현재 프레임을 서버에 저장하여 향후 모델 학습을 위한 데이터셋으로 활용할 수 있다. 3. 웹 기반 비상 인증 시스템 (Web Keypad Auth) 생체 인식이 불가능한 예외 상황을 고려하여 웹 기반의 가상 키패드를 구축하였다. 이는 JavaScript와 Flask API 연동으로 작동하며, 사용자가 올바른 PIN 번호를 입력할 경우 안면 인식 프로세스를 우회하여 즉시 도어락을 개방하는 강제 개방(Force Unlock) 로직을 수행함으로써 시스템의 가용성을 극대화하였다.		
기대효과 (200자내외)	본 프로젝트는 딥러닝 기반의 안면 인식 기술을 도입함으로써, 물리적 열쇠의 분실이나 비밀번호 유출과 같은 기존 보안 시스템의 구조적 취약점을 원천적으로 차단하고 보안 무결성을 대폭 강화하였다. 특히 YOLO 모델을 활용한 정밀한 객체 탐지와 거리 비례 인식 알고리즘을 적용하여 타인 오인식 및 오작동 가능성을 최소화하였으며, 사용자가 별도의 물리적 접촉 없이 카메라를 응시하는 행위만으로 잠금이 해제되는 핸드프리(Hands-free) 환경을 구현하여 사용자의 편의성을 극대화하였다. 더불어, 안면 인식이 불가능한 환경이나 미등록 방문자 발생 등 예외 상황을 대비하여 웹 서버(Flask) 기반의 비상 키패드 기능을 결합함으로써 시스템의 가용성을 보장하였다. 결과적으로 본 시스템은 엣지 디바이스의 생체 인식 기술과 웹 인터페이스의 확장성을 유기적으로 결합하여, 어떠한 환경에서도 안정적이고 유연한 출입 통제가 가능한 지능형 하이브리드 보안 솔루션을 완성하였다.		
Keywords	AIoT	Face Recognition	YOLOv8
	Raspberry Pi 5	Flask	얼굴 인식

목 차

I. 서론	1
1. 연구 목적 및 목표	2
2. 본 논문 연구의 기여 내용	2
3. 보고서 구성 내용	2
II. 관련 연구	3
1. 이전 연구	3
2. 국내외 시장 동향	3
III. 기본 이론	4
1. YOLO	4
2. Flask	4
3. 딥러닝 기반 안면 인식 및 임베딩	4
IV. 설계 시스템 구성	5
1. 하드웨어 구성	5
2. 소프트웨어 및 시스템 아키텍처	5
3. 주요 알고리즘	7
V. 동작 결과 및 결과 분석	10
1. 주요 동작 결과	10
2. 계획 대비 일정표	11
3. 성능 평가표	11
3. 결과 분석	11
VI. 결론	12
1. 결론 요약	12
2. 앞으로 추가할 연구	12
VII. 참고 문헌	14
VIII.	14

I. 서론

1-1. 연구 목적

현대 사회에서 주거 침입 범죄에 대한 우려가 증가함에 따라, 주거 보안의 첫 단계인 도어락 시스템의 중요성이 대두되고 있다. 그러나 기존의 물리적 열쇠(Key) 방식은 분실 및 복제의 위험이 존재하며, 디지털 도어락의 비밀번호 입력 방식은 잔류 지문 노출이나 타인에게 쉽게 공유될 수 있다는 보안적 취약점을 안고 있다.

이에 본 연구는 사물인터넷(IoT) 기술과 딥러닝 기반의 컴퓨터 비전 기술을 융합하여 이러한 문제점을 해결하고자 한다. 고성능 엣지 디바이스인 Raspberry Pi 5를 기반으로 YOLOv8 객체 탐지 알고리즘을 이식하여, 별도의 접촉 없이 사용자를 실시간으로 식별하고 출입을 통제하는 '지능형 AIoT 안면인식 도어락 시스템'을 구현하는 것을 주된 목적으로 한다. 이를 통해 보안성을 강화함과 동시에 사용자의 편의성을 극대화한 스마트 홈 보안 솔루션을 제시하고자 한다.

(1) 연구 목표

본 연구는 물리적 열쇠나 비밀번호 입력 방식이 가지는 분실, 복제, 위생상의 한계를 극복하기 위해 딥러닝 기반의 비접촉식 안면인식 도어락 시스템을 개발하는 것을 주된 목적으로 한다.

특히 클라우드에 의존하지 않고 Raspberry Pi 5 엣지 디바이스 내에서 독립적으로 연산을 수행하는 On-Device AI 시스템을 구축하여, 네트워크 지연 없는 실시간 반응성을 확보하고 개인 생체 정보의 외부 유출을 방지한다. 최종적으로는 인공지능(AI), 사물인터넷(IoT), 웹 기술(Web)을 유기적으로 결합하여 보안성과 사용자 편의성을 동시에 만족하는 지능형 스마트 홈 솔루션을 제시하고자 한다.

(2) 기술적 목표

1. **강건한 안면 인식 구현:** YOLOv8 모델을 엣지 디바이스에 최적화하고, 거리 비례 면적(Face Area Ratio) 필터링을 적용하여 원거리 오인식을 방지하는 고정밀 식별 알고리즘을 구현한다.
2. **정밀 임베디드 제어:** GPIO 순차 제어를 통한 스텝 모터(Stepper Motor) 구동으로 물리적 잠금 장치를 정밀하게 제어하며, LCD와 부저를 통해 직관적인 사용자 피드백을 제공한다.
3. **하이브리드 웹 관제 시스템:** Flask 기반의 웹 서버를 구축하여 실시간 스트리밍 및 학습 데이터 캡처 기능을 제공하고, 안면 인식 불가 시 PIN 번호(키패드)로 대체 인증하는 Fail-Safe 환경을 완성한다.

1-2. 본 논문 연구의 기여 내용

- 첫째, 거리 인식이 결여된 2D 객체 탐지 모델의 한계를 알고리즘적으로 개선하였다. 기존의 단순 YOLO 기반 시스템은 거리에 상관없이 객체를 탐지하여, 문 앞이 아닌 원거리에 있는 행인을 오인식하는 보안 취약점이 존재했다. 본 연구는 Bounding Box의 면적 비율(Area Ratio)을 실시간으로 산출하여 일정 거리 이내의 사용자만 선별적으로 인증하는 '거리 감응형 필터링 알고리즘'을 제안하고 구현함으로써, 단일 카메라 만으로도 깊이(Depth) 인식 부재 문제를 효과적으로 보완하였다.
- 둘째, 엣지 컴퓨팅과 웹 기술을 융합하여 독립적이고 지속 가능한 데이터 파이프라인을 구축하였다. 외부 클라우드 서버에 의존하지 않고 Raspberry Pi 5 온디바이스(On-device) 환경에서 추론과 제어를 모두 수행함으로써 개인정보 보호와 네트워크 지연 문제를 해결하였다. 또한, Flask 웹 인터페이스를 통해 현장 데이터를 즉시 캡처(Capture)하고 학습 데이터로 변환할 수 있는 순환 구조를 설계하여, 모델의 유지보수와 성능 고도화가 용이한 MLOps 기초 환경을 마련하였다.
- 셋째, 생체 인식의 환경적 제약을 극복하는 하이브리드 페일 세이프(Fail-Safe) 시스템을 정립하였다. 조도 부족이나 마스크 착용 등 안면 인식이 불가능한 예외 상황(Edge Case)에서 시스템이 무용지물이 되는 것을 방지하기 위해, 웹 기반의 가상 키패드 인증 방식을 결합하였다. 이는 단순한 기능 추가를 넘어, 생체 인증의 편의성과 비밀번호 인증의 신뢰성을 상호 보완하는 이중 안전장치 구조를 실증하였다는 데에 의의가 있다.

1-3. 보고서 구성

본 보고서는 총 6장으로 구성된다.

- 제2장에서는 생체 인식 기반의 보안 시스템 관련 연구 동향과 스마트 홈 도어락 시장의 기술적 현황을 분석한다.
- 제3장에서는 본 시스템의 핵심이 되는 YOLOv8 객체 탐지 모델, 안면 임베딩(Face Embedding) 기술, 그리고 Flask 웹 프레임워크와 OpenCV 영상 처리의 이론적 배경을 기술한다.
- 제4장에서는 Raspberry Pi 5를 기반으로 한 하드웨어 회로 구성과 멀티스레딩 및 거리 비례 필터링 알고리즘이 적용된 소프트웨어 아키텍처 설계를 상세히 설명한다.
- 제5장에서는 시스템의 실제 구현 모습과 시나리오별 동작 결과(인증 성공/실패, 거리 제어 등)를 제시하고, 시스템의 성능을 분석 및 고찰한다.
- 제6장에서는 본 연구의 결론을 도출하고, 현재 식별된 오인식 문제의 개선 방안과 보안성 강화를 위한 향후 연구 과제를 제언한다.

II. 관련 연구

2-1. 이전 연구

(1) 도어락 시스템의 진화와 생체 인식 기술

- 기존의 도어락 시스템은 열쇠를 사용하는 기계식 방식에서 비밀번호나 RFID 카드를 사용하는 디지털 방식으로 발전해왔다. 그러나 이러한 접촉식 매체는 분실, 복제, 타인 양도 등의 보안 취약점을 내재하고 있다. 이에 따라 고유한 신체적 특징을 이용하는 생체 인식(biometric) 기술이 대안으로 부상하였으며, 그 중에서도 비접촉 방식인 안면 인식 기술은 사용자의 거부감이 적고 편의성이 높아 스마트 홈 보안의 핵심 기술로 연구되고 있다.

(2) 객체 탐지 알고리즘의 발전과 YOLO 모델

- 초기 안면 인식 연구는 Harr Cascades나 HOG와 같은 핸드 크래프트 특징 추출 방식을 주로 사용하였다. 이는 연산량이 적다는 장점이 있으나, 조명 변화나 얼굴 각도 변화에 취약하다는 한계가 있었다. 이후 CNN(Convolutional Neural Networks) 기반의 딥러닝 기술이 등장하며 인식률이 비약적으로 상승하였으나, R-CNN 계열과 같은 2단계(Two-stage) 탐지 모델은 연산 속도가 느려 실시간 시스템 적용에 어려움이 있었다. 이에 본 연구에서는 속도와 정확도의 균형이 우수한 1단계(One-stage) 탐지 모델인 YOLO(You Only Look Once) 알고리즘을 채택하여, 실시간성이 요구되는 도어락 시스템에 최적화된 객체 탐지를 수행하고자 한다.

(3) 기존 연구의 한계 및 차별성

- 선행된 라즈베리 파이 기반의 DIY 도어락 연구들은 단순히 얼굴 탐지 유무(Face Detected)만을 트리거로 사용하여, 문 앞이 아닌 원거리에 위치한 사람을 오인식하거나 배경의 사진을 얼굴로 인식하는 등의 오작동 사례가 빈번했다. 또한, 안면 인식이 실패했을 때를 대비한 대체 인증 수단(Backup Auth)이 부재하여 시스템 가용성이 떨어지는 문제점이 있었다. 이에 본 연구는 YOLOv8-face를 기반으로 하되 얼굴 면적 비율(Face Area Ratio) 분석 알고리즘을 추가하여 근접한 사용자만 선별적으로 인식하도록 개선하였으며, 웹 기반의 가상 키패드를 융합하여 기존 연구의 한계점인 보안성과 안정성을 동시에 해결하고자 한다.

2-2. 국내외 시장 동향

- (1) 국외 시장: 글로벌 스마트 도어락 시장은 2025년 약 31억 9,000만 달러에서 52억 9,000만 달러 규모에 이를 것으로 전망되며, 연평균 성장률(CAGR)은 약 12%~21%에 달하는 고성능 시장이다. 특히 전체 스마트 도어락 시장 중 생체 인식(Biometric) 제품군이 약 54%의 점유율을 차지하며 시장 성장을 주도하고 있다. 2025년의 핵심 트렌드로는 Matter 1.4와 같은 개방형 표준 확산과 더불어, 지문 중심에서 AI 기반 안면 인식 및 손바닥 정맥 등으로 인증 수단이 다변화되고 있는 추세이다.

- (2) 국내 시장: 국내 보안 시장은 1인 가구 증가와 주거 범죄에 대한 경각심이 높아짐에 따라 '홈 보안 솔루션'의 보급이 확대되고 있다. 특히 2025년 주요 보안 트렌드로 '고도화된 얼굴 인식 솔루션'이 각광받고 있으며, 팬데믹 이후 정착된 비접촉

(Contactless) 생체 인증에 대한 소비자 선호도가 매우 높다. 최근 IP 카메라 해킹 사고 등으로 인해 영상 데이터 유출에 대한 우려가 커지면서(설문 응답자의 75%가 위험 체감), 단순한 연결성을 넘어 보안 인증을 받은 신뢰성 있는 제품에 대한 수요가 급증하고 있다.

III. 기본 이론

3-1. YOLO 알고리즘

- YOLO는 객체 탐지를 하나의 회귀 문제로 정의하여 이미지 전체를 한 번의 신경망 통과만으로 처리하는 획기적인 알고리즘이다 [1]. 본 연구에서는 Ultralytics사에서 발표한 최신 모델인 YOLOv8n을 사용하여 [2], 라즈베리 파이 환경에서의 실시간성을 확보하였다.
- 탐지 및 거리 필터링: 입력 이미지를 그리드로 분할하여 객체의 Bounding Box와 클래스 확률을 동시에 예측한다. 특히 단순 탐지를 넘어 BBox의 면적 비율(Area Ratio)을 실시간으로 계산하여, 일정 거리 이내에 근접한 유효 사용자만을 선별적으로 인식하도록 알고리즘을 최적화하였다.

3-2. Flask (웹 프레임 워크)

- Flask는 필수 기능만을 제공하여 가볍고 유연한 Python 기반의 마이크로 웹 프레임워크(Micro Web Framework)이다. 메모리 자원이 중요한 임베디드 시스템에 적합하도록 불필요한 오버헤드를 줄여 시스템 부하를 최소화하였다.
- 스트리밍 및 하드웨어 제어: HTTP Multipart Response 방식을 활용하여 지연 없는 MJPEG 비디오 스트리밍을 구현하였으며, REST API 라우팅을 통해 웹 인터페이스의 요청(인증 시도, 강제 개방)과 하드웨어 제어(GPIO, 카메라) 간의 신호를 실시간으로 중계한다.

3-3. 딥러닝 기반 안면 인식 및 임베딩 (Face Embedding)

3-3-1. 안면 임베딩(Face Embedding)의 개념

- 안면 임베딩이란 고차원의 얼굴 이미지를 저차원의 실수 벡터로 변환하는 기술이다. Google의 FaceNet 연구에 따르면, 딥러닝 모델을 통해 추출된 128차원의 특징 벡터는 유클리드 거리(Euclidean Distance)를 통해 동일인 여부를 높은 정확도로 판별할 수 있다 [3]. 본 시스템은 이를 기반으로 한 dlib 라이브러리를 활용하여 신원 검증을 수행한다 [4].

3-3-2. 거리 기반 유사도 측정 (Metric Learning)

- 추출된 임베디드 벡터가 등록된 사용자인지 판별하기 위해 유클리드 거리(Euclidean Distance) 알고리즘을 사용한다. 두 얼굴 벡터 A 와 B 사이의 거리가 가까울수록(0에 수렴할수록) 동일인일 확률이 높고, 멀어질수록 타인일 확률이 높다. 본 시스템에서는 이 거리 값을 계산하여 사전에 설정된 임계값(Threshold, 본 연구에서는

0.65) 이하일 경우에만 동일인으로 간주하여 잠금을 해제하는 인증 로직을 수행한다.

IV. 설계 시스템 구성

4-1. 하드웨어 구성

1) Raspberry Pi 5

- AI 모델(YOLOv8)의 추론과 영상 처리, Flask 웹 서버 구동을 동시에 수행하는 중앙 제어 장치이다. 8GB의 넉넉한 RAM과 향상된 프로세서 성능을 통해, 엣지(Edge) 환경에서도 지연 없는 실시간 얼굴 인식과 멀티스레딩 작업을 안정적으로 보장한다.

2) USB Web Camera

- 실시간 출입자 영상 데이터를 수집하기 위한 시각 입력 장치이다. OpenCV 라이브러리와 높은 호환성을 가지며, 시스템의 부하를 최소화하면서도 얼굴 식별에 충분한 해상도(640 × 480)의 영상을 지속적으로 메인 컨트롤러에 전송한다.

3) Stepper Motor & Driver

- 도어락의 물리적 잠금(Lock) 및 해제(Unlock) 동작을 수행하는 정밀 액추에이터이다. 일반 DC 모터와 달리 회전 각도를 정확하게 제어할 수 있어 문 개폐 상태를 확실하게 유지하며, 전용 드라이버 모듈을 통해 GPIO 신호를 안정적인 구동 전류로 변환하여 모터를 제어한다.

4) RGB LED Module

- 사용자에게 현재 보안 시스템의 상태를 직관적으로 전달하는 시각적 알림 장치이다. 적색(잠금/실패), 녹색(개방/성공), 청색(인식 중/대기)의 색상 변화를 통해 사용자가 화면을 보지 않고도 인증 진행 상황을 파악할 수 있도록 한다.

5) Buzzer

- 시스템 조작에 따른 청각적 피드백을 제공하는 음향 출력 장치이다. PWM(펄스 폭 변조) 제어를 통해 단순한 경고음뿐만 아니라 성공, 실패, 처리 중 등 상황에 따라 서로 다른 주파수와 패턴의 알림음을 출력하여 사용자 경험(UX)을 향상시킨다.

4-2. 소프트웨어 및 시스템 아키텍처

(1) 소프트웨어 구성

- Language: Python 3.11.9 – AI 라이브러리 호환성 및 안정성 확보를 위해 3.11.9 버전 사용
- AI Engine: Ultralytics YOLOv8n-face – 얼굴 탐지에 특화된 경량화 모델 사용
- Vision Lib: OpenCV 4.x – 이미지 전처리 및 실시간 영상 스트림 처리
- Web F/W: Flask 3.x – 경량 REST API 서버 및 MJPEG 스트리밍 구현
- H/W Control: GPIO Zero – 스텝 모터, LED, 부저의 정밀제어

(2) 시스템 아키텍처

1. AI Inference & Control Thread

- 역할: 카메라로부터 영상을 획득(Capture)하고 AI 추론 및 하드웨어 제어를 수행하는 핵심 루프이다.
- 동작: OpenCV를 통해 최신 프레임을 실시간으로 읽어와 YOLOv8 모델에 전달한다. 얼굴 탐지 결과에 따라 인증 로직을 수행하고, 서보 모터와 LED 상태를 즉각적으로 갱신하여 딜레이 없는 물리적 보안 기능을 제공한다.

2. Flask Web Server Thread

- 역할: 사용자 인터페이스(Web UI)와 시스템 간의 비동기 통신을 담당한다.
- 동작: 메인 로직과 독립적으로 동작하며, 사용자의 인증 시도(/request_auth)나 강제 개방(/force_unlock) 요청을 수신하여 전역 상태 변수(State Variable)를 갱신한다. 또한, Generator 패턴을 통해 처리된 영상 스트림을 웹 클라이언트로 송출한다.

3. Async Feedback Thread

- 역할: 영상 처리 성능에 영향을 주지 않고 즉각적인 시청각 알림을 제공한다.
- 동작: 부저(Buzzer) 출력과 같이 시간이 소요되는(Blocking) 작업은 별도의 일회성 스레드(Transient Thread)로 분리하여 실행한다. 이를 통해 알림음이 재생되는 동안에도 영상 분석이 멈추거나 프레임이 끊기는 현상을 방지하였다.

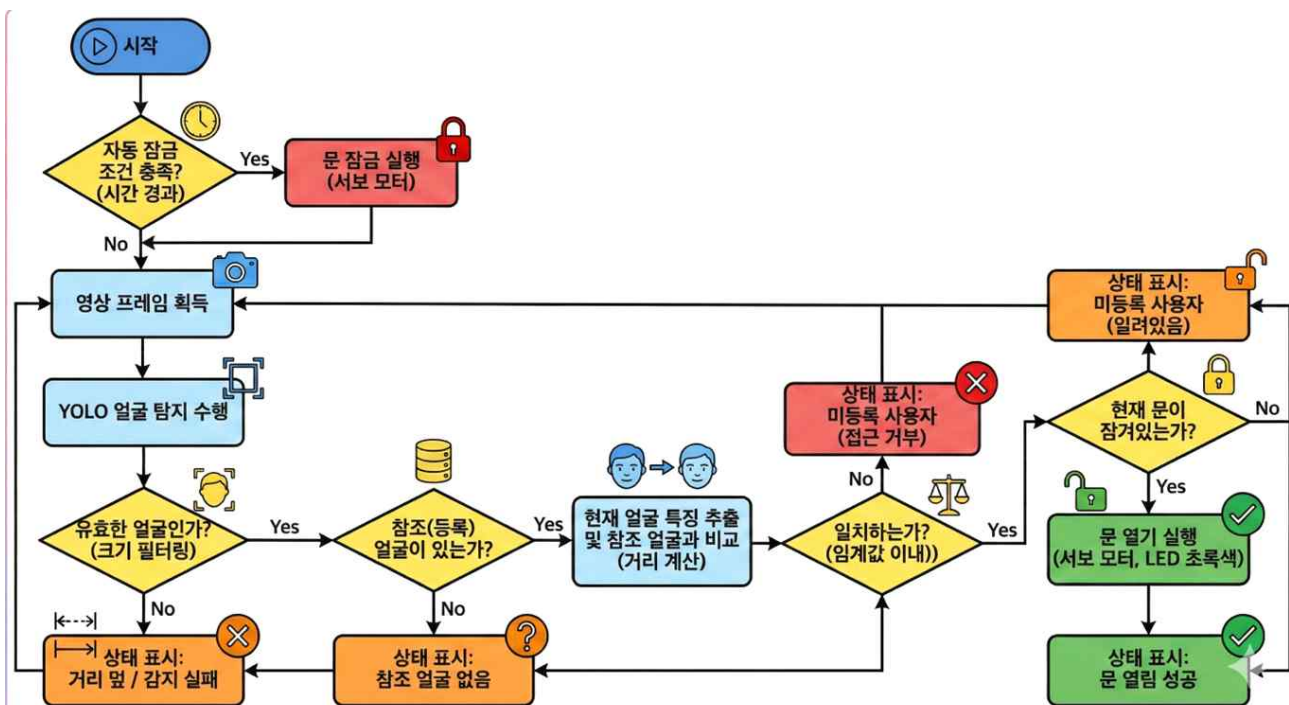


그림 1 시스템 동작 알고리즘

4-2. 주요 알고리즘

1) 거리 비례 필터링 알고리즘

- 단안 카메라(Mono Camera)를 사용하는 환경에서 객체와의 거리를 추정하기 위해, 바운딩 박스(Bounding Box)의 면적 비율을 활용하는 알고리즘을 제안하였다. YOLO 모델이 반환하는 좌표 (x_1, y_1, x_2, y_2) 를 기반으로 객체의 면적(S_{obj})과 전체 프레임 면적(S_{frame})의 비율(R)을 실시간으로 산출한다.

$$S_{obj} = (x_2 - x_1) \times (y_2 - y_1)$$

$$R = \frac{S_{obj}}{S_{frame}}$$

- 이때 산출된 비율 R 이 임계값(Threshold $\theta=0.01$) 미만일 경우, 시스템은 해당 객체를 원거리에 위치한 배경 요소로 간주하여 인증 후보군에서 즉시 제외한다. 이를 통해 문 앞의 실제 사용자만 정확하게 인식하는 공간적 필터링을 수행한다.

```
# [1] 면적 비율 계산 함수
def bbox_area_ratio(x1, y1, x2, y2, frame_w, frame_h):
    bw = max(0, x2 - x1)
    bh = max(0, y2 - y1)
    box_area = bw * bh
    frame_area = max(1, frame_w * frame_h)
    return box_area / frame_area

# 계산된 area_r(비율)이 설정값(0.01)보다 커야만 인식 후보(best_info)로 등록됨
if area_r >= MIN_FACE_AREA_RATIO:
    if best_info is None or area_r > best_info[-1]:
        best_info = (x1, y1, x2, y2, conf, class_name, area_r)
```

2) 하이브리드 신원 검증 알고리즘

탐지된 얼굴이 인가된 사용자인지 판별하기 위해, Class Classification과 Feature Embedding을 결합한 검증 로직을 수행한다.

- 1차 권한 검사 (Authorization Check): YOLO가 예측한 클래스 이름(C_{pred})이 사전에 정의된 인가 사용자 목록($L_{auth}=\{HJH, KTJ, \dots\}$)에 포함되는지 확인한다. 목록에 없는 경우(Unknown) 즉시 'UNAUTHORIZED'로 판정하여 리소스를 절약한다.
- 2차 정밀 매칭 (Embedding Matching): 권한 검사를 통과한 경우, 얼굴 영역(ROI)을 크롭(Crop)하여 128차원의 특징 벡터(V_{ip})를 추출한다. 이를 사전에 등록된 참조 벡터(V_{ref})와 유클리드 거리(Euclidean Distance)로 비교한다.

$$Distance(V_{ip}, V_{ref}) = \sqrt{\sum_{i=1}^{128} (V_{ip}^{(i)} - V_{ref}^{(i)})^2}$$

- 계산된 거리가 설정된 임계값(0.65) 이하일 때만 최종적으로 잠금 해제 신호를 발생시킨다.

```

# [1] 1차 권한 검사 (Class Classification)
# 탐지된 이름이 허용된 사용자 목록(AUTHORIZED_USERS)에 없으면 즉시 차단
if class_name not in AUTHORIZED_USERS:
    cv2.putText(final_display, "UNAUTHORIZED", (x1, y1-30), cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 0, 255), 2)
# [2] 2차 정밀 매칭 (Embedding Matching)
# face_recognition 라이브러리를 통해 128차원 벡터 거리 계산
def match_reference(face_bgr, class_name_now=None):
    # ... (중략)
    if FACE_COMPARE_AVAILABLE and reference_encoding is not None:
        # 벡터 변환 및 거리 계산
        rgb = cv2.cvtColor(face_bgr, cv2.COLOR_BGR2RGB)
        encs = face_recognition.face_encodings(rgb)

        # 유클리드 거리(dist)가 임계값(0.65) 이하인지 확인
        dist = face_recognition.face_distance([reference_encoding], encs[0])[0]
        ok = dist <= FACE_DISTANCE_THRESHOLD
        return ok, f"dist={dist:.3f}"

```

3) 요청 기반 비동기 감지 루프

시스템의 전력 소모를 줄이고 오작동을 방지하기 위해, 상시 감지가 아닌 이벤트 트리거(Event-Triggered) 방식의 알고리즘을 적용하였다.

1. 대기 모드 (Idle): 평상시에는 카메라 스트리밍만 유지하며, 고부하 작업인 AI 추론은 수행하지 않는다.
2. 트리거 및 활성화: 웹 인터페이스의 '인증 요청' 버튼이나 외부 신호가 감지되면 is_auth_active 플래그가 활성화된다.
3. 타임아웃 제어: 활성화 시점부터 10초(AUTH_TIMEOUT) 동안만 추론 루프가 동작하며, 시간 내에 인증에 성공하지 못하면 자동으로 대기 모드로 복귀하여 보안성을 확보한다.

```

# [1] 트리거 활성화 (Flask Route)
@app.route('/request_auth')
def request_auth():
    global is_auth_active, auth_start_time
    # ...
    is_auth_active = True          # 플래그 활성화
    auth_start_time = time.time()  # 타이머 시작
    return jsonify({'result': 'started'})
# [2] 타임아웃 제어 (run_detection_thread 내부)
if is_auth_active:
    # 현재 시간과 시작 시간의 차이가 10초(AUTH_TIMEOUT)를 넘으면 강제 종료
    if current_time - auth_start_time > AUTH_TIMEOUT:
        add_log("🕒 인증 시간 초과 (실패)")
        is_auth_active = False     # 플래그 비활성화 (대기 모드 복귀)

```

4) 멀티스레드 기반 상태 동기화

Flask 웹 서버와 AI 추론 엔진 간의 데이터 충돌을 방지하고 실시간성을 보장하기 위해 다음과 같은 동기화 기법을 적용하였다.

- Frame Locking: 웹으로 송출되는 영상 데이터(output_frame)에 threading.Lock과 Condition 객체를 적용하여, AI 스레드가 이미지를 쓰고 있는 도중에 웹 스레드가 읽으려 할 때 발생하는 티어링(Tearing) 현상을 방지하였다.
- Event Handling: threading.Event 객체를 사용하여 캡처 요청(capture_event)이나 시스템 종료 신호(stop_event)를 스레드 간에 안전하게 전달하는 비동기 통신 구조를 확립하였다.

```
# [1] 동기화 객체 선언
frame_lock = threading.Lock()
frame_cond = threading.Condition(frame_lock)

# [2] AI 스레드에서 쓰기 (Write)
with frame_cond:
    output_frame = final_display
    frame_cond.notify_all() # 대기 중인 웹 스레드에 알림

# [3] 웹 스레드에서 읽기 (Read)
def generate_frames():
    # ...
    with frame_cond:
        if output_frame is None:
            frame_cond.wait(timeout=1.0) # 새 프레임이 올 때까지 대기
        frame = output_frame
```

V. 동작 결과 및 결과 분석

5-1. 주요 동작 결과

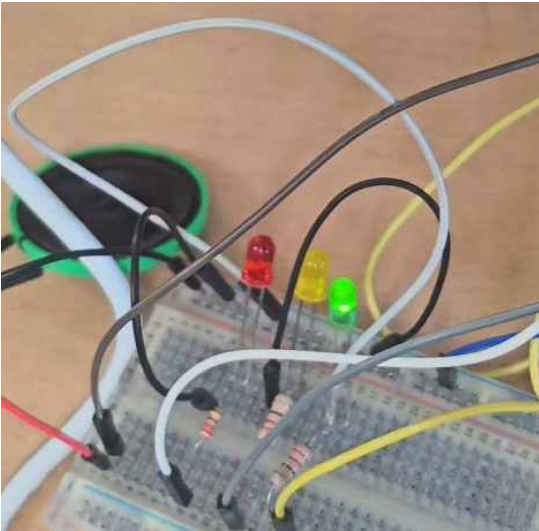


그림 3 문열림 상태 (초록 LED 점등)

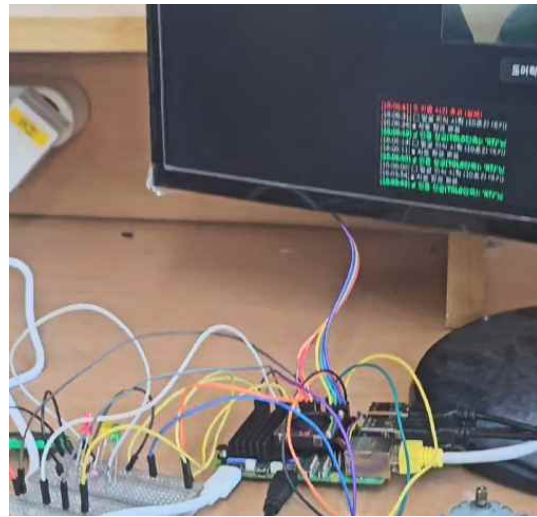


그림 2 문닫힘 상태 (붉은 LED 점등)



그림 4 Keypad 작동



그림 5 기본화면 및 사진 촬영

5-2. 계획 대비 일정표

	11월												12월																			
	1주차				2주차				3주차				4주차				1주차				2주차				3주차				4주차			
	11.3-11.9				11.10-11.16				11.17-11.23				11.24-11.30				12.1-12.7				12.8-12.14				12.15-12.21				12.22			
요구사항 정의																																
모델학습																																
시스템 아키텍처																																
하드웨어 구성 및 라즈베리파이 연동																																
센서, 제어 로직 개발																																
통합 테스트 검증																																
발표 자료 제작																																
발표 준비																																

파란색은 추가된 일정, 회색은 계획대로 진행된 일정

5-3. 성능 평가표

평가항목	제시한 성능 사양	만족(25점)	부분만족(10점)	불만족(5점)	총점
얼굴 인식 지원	카메라로 사람 얼굴인식이 잘 되는가?	사람 얼굴을 구분할 수 있다.	얼굴 인식만 된다.	얼굴 인식이 안된다.	10
도어 개폐장치	개폐장치가 정상적으로 잘 작동하는가?	문이 원활하게 열린다.	문이 간헐적으로 열린다.	문이 안 열린다.	25
응답 속도	얼마나 문이 빨리 열리는가?	5초 이내로 소요된다.	10초 이내로 소요된다.	20초 이상 소요된다.	25
인식 범위	여러 위치에서도 인식되는가?	거리에 상관없이 잘 인식된다.	특정범위 내에서만 인식된다.	인식이 되지 않는다.	25
총점	85점				

5-4. 결과 분석

- 1) 안면 인식 정확도 및 한계 조원 4인에 대한 식별 성능은 우수하였으나, 특정 외부인을 조원으로 오인식하는 현상이 간헐적으로 발생하였다. 이는 사용 편의성을 위해 설정한 유사도 임계값(0.65)이 다소 관대하여, 이목구비가 비슷한 타인을 허용했기 때문으로 분석된다.
- 2) 거리 제어 알고리즘 검증 얼굴 면적 비율이 1% 미만일 때 인식을 차단하는 거리 필터링 로직은 매우 효과적이었다. 이를 통해 원거리에 있는 사람을 배경으로 처리하여 불필요한 작동을 방지하고, 문 앞의 유효 사용자만 정확히 감지하였다.
- 3) 시스템 통합 성능 Raspberry Pi 5의 멀티스레딩 환경에서 AI 추론, 웹 스트리밍, 하드웨어 제어가 지연 없이 병렬 처리됨을 확인하였다. 특히 웹에서의 강제 개방 명령과 하드웨어(모터/LED) 반응 속도가 실시간으로 동기화되었다.

VI. 결론

6-1. 결론 요약

본 연구에서는 고성능 엣지 디바이스인 Raspberry Pi 5와 경량화된 딥러닝 모델 YOLOv8n-face를 결합하여, 외부 네트워크 의존성 없이 독립적으로 구동되는 지능형 안면 인식 도어락 시스템을 설계 및 구현하였다.

- 첫째, 기존의 2차원 객체 탐지 기술이 가진 공간적 한계를 극복하기 위해 거리 비례 필터링(Area Ratio Filtering) 알고리즘을 도입하였다. 이를 통해 단순히 얼굴을 찾는 것을 넘어, 카메라로부터 일정 거리(약 1m) 이내에 접근한 유효 사용자만을 선별적으로 인식하는 로직을 구현함으로써, 배경의 행인이나 원거리 객체에 의한 오작동을 원천적으로 차단하였다.
- 둘째, 하이브리드 인증 메커니즘을 통해 보안성과 편의성의 균형을 확보하였다. YOLO를 이용한 1차 객체 탐지와 Face Embedding을 이용한 2차 정밀 신원 검증 과정을 거쳐, 등록된 조원(HJH, KTJ, UY, HKM)을 정확하게 식별하는 데 성공하였다. 또한, 인식 실패 시 즉각적인 시청각 피드백(Red LED, Buzzer)을 제공하고 웹 인터페이스를 통한 비상 개방 경로를 마련하여 시스템의 가용성을 높였다.
- 셋째, 멀티스레딩 아키텍처를 적용하여 단일 보드 상에서 영상 처리, 웹 서버 구동, 하드웨어 제어를 병렬로 처리함에 있어 지연 없는 실시간성을 입증하였다. 결과적으로 본 연구는 고가의 상용 보안 시스템을 대체할 수 있는 저비용·고효율의 엣지 AI 보안 솔루션의 기술적 타당성을 제시하였다.

6-2. 앞으로 추가로 할 연구

1. 타인 수락률(FAR) 개선을 위한 임계값 최적화

현재 시스템은 사용자 편의를 위해 유사도 임계값(FACE_DISTANCE_THRESHOLD)을 0.65로 설정하였다. 이로 인해 이목구비가 유사한 특정 외부인을 조원으로 오인식하는 사례가 확인되었다. 향후 연구에서는 적응형 임계값(Adaptive Thresholding) 알고리즘을 도입하여, 주변 조도나 얼굴 각도에 따라 임계값을 동적으로 조절하거나, 기본 임계값을 0.55 이하로 하향 조정하여 보안 강도를 높이는 최적화 작업이 필요하다.

2. 위변조 방지(Anti-Spoofing) 기술 도입

현재의 2D 이미지 기반 인증은 고화질 사진이나 동영상을 이용한 스푸핑(Spoofing) 공격에 취약할 수 있다. 이를 보완하기 위해 Liveness Detection(생체 반응 탐지) 기술을 추가해야 한다.

- 하드웨어적 접근: 적외선(IR) 카메라나 ToF(Time of Flight) 센서를 추가하여 얼굴의 깊이(Depth) 정보를 획득, 평면 사진과 입체적인 실제 얼굴을 구분한다.
- 소프트웨어적 접근: 눈 깜빡임(Eye Blinking)이나 입 모양의 미세한 움직임을 감지하는 알고리즘을 추가하여 살아있는 사용자인지를 검증한다.

3. 사용자 관리 데이터베이스(DB) 구축

현재는 소스 코드 내에 사용자 목록(AUTHORIZED_USERS)이 하드코딩되어 있고, 참조 이미지를 파일 시스템에 덮어쓰는 방식으로 관리하고 있다. 향후에는 SQLite와 같

은 경량 데이터베이스를 연동하여, 웹 인터페이스에서 관리자가 코드를 수정하지 않고도 사용자를 손쉽게 등록, 삭제, 수정할 수 있는 사용자 관리 시스템(User Management System)으로 확장해야 한다.

VII. 참고문헌

- [1] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), 2016, pp. 779–788.
- [2] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLO," Ultralytics, Jan. 2023. [Online]. Available: <https://github.com/ultralytics/ultralytics>.
- [3] F. Schroff, D. Kalenichenko, and J. Philbin, "FaceNet: A Unified Embedding for Face Recognition and Clustering," in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), 2015, pp. 815–823.
- [4] D. E. King, "Dlib-ml: A Machine Learning Toolkit," J. Mach. Learn. Res., vol. 10, pp. 1755–1758, 2009.

VIII. 부록

1. 회의록

제1차 회의록

일시: 2025년 11월 18일 금요일 17시 00분 ~ 18시 00분

장소: x (화상회의)

참석자: 허진혁, 권태정, 이준영, 한경미

안건: 프로젝트 주제 확정

1. 프로젝트 주제 선정

주제를 선정하기 위해 미리 주제를 생각해 오기로 했었고 다들 여러 아이디어를 내고 논의한 결과, 초기에는 AI 스마트팜이 선정되었으나 AIoT라는 주제를 더 잘 살리기 위해 얼굴 인식 도어락을 최종 주제로 선정하였다. 해당 프로젝트는 라즈베리파이에 카메라를 연결하여 등록된 특정 사람이 인식되면 자동으로 문이 열리는 것을 목표로 하는 프로젝트이며, 기존 도어락과의 차이점이라면 YOLOv8n-face 모델을 이용하여 사람의 얼굴을 학습해 얼굴을 열쇠로 사용하여 자동으로 문이 열리는 기능을 핵심으로 정하였다.

2. 차기 회의 일정

12월 11일 오후에 회의를 진행하기로 하였다.

제 2차 회의록

일시: 2025년 12월 11일 목요일 15시 00분 ~ 18시 00분

장소: 정보 814

참석자: 허진혁, 권태정, 이준영, 한경미

안건: 개발 진행 상황 공유 및 테스트

1. 진행 상황

그간 기간동안 현재 각자 맡은 부분에 대해서 얼마나 진행이 되었는지 상황을 공유하였다. 얼굴인식은 저조한 학습률로 인해 난항을 겪고 있었고, 우선적으로 Opencv를 이용한 객체 검출 및 사진 캡처 기능은 성공적으로 완료하였다.

2. 모델 학습

꽤 많은 시간을 모델 학습을 위해 투자하였으나 Teachable Machine의 성능이 생각보다 더 좋지 않았다. Edge Impulse 방식으로 모델 학습을 다시 시도하였으나 이 결과 또한 좋지 않아서 방법을 찾다가 결국 YOLO의 자체 기능을 이용해 학습을 시도하기로 하였다.

3. 코드 정리

현재 작성된 코드를 하나로 병합하는 과정을 거쳤다. 각자 개별 환경에서 진행한 터라 라이브러리와 파이썬 버전으로 인한 충돌이 너무 발생해서 버전을 하나로 확립하고 재설정하였다.

4. 차기 회의 일정

곧 시험 기간이기 때문에 시험이 끝나고 마무리 하자고 의견을 모았고 12월 19일에 마지막 준비를하기로 하였다.

제 3차 회의록

일시: 2025년 12월 19일 금요일 10시 00분 ~ 16시 00분

장소: 정보 814, 프라임실

참석자: 허진혁, 권태정, 이준영, 한경미

안건: 최종 기능 점검, 오류 수정

1. 최종 점검

완성된 코드들은 현재 윈도우 환경에서 테스트 했었고 해당 코드들을 라즈베리파이 환경에서 원활하게 실행될 수 있게끔 수정하였다. dlib 설치와 face_recognition 설치에 난항을 겪었지만 다행히 해결할 수 있었고, 윈도우 환경에서 미처 잡지 못한 버그와 문제들을 수정하였다.

2. 오류 수정

한 가지 간과했던 사실은 우리의 개인 PC의 성능이 좋아서 아무리 하드웨어 자원을 잡아먹어도 아무런 문제없이 실행 되었지만 라즈베리파이는 자원이 한정되어있기 때문에 동일한 코드라도 라즈베리 환경에서는 상당한 부하가 걸렸다. 이에 맞춰 코드를 경량화하고 최대한 부하가 덜 걸리게끔 수정하며 라즈베리파이의 공간을 늘리기 위해 필요없는 파일이나 라이브러리는 지우는 식으로 확보하였다.

3. 개선 아이디어

프로젝트를 더 발전 시키기 위한 아이디어로서 로그를 데이터베이스 차원에서 관리하는 기능, 그리고 모델 학습의 자동화, 휴대폰 어플과의 연동 등 다양한 아이디어가 제안 되었다.

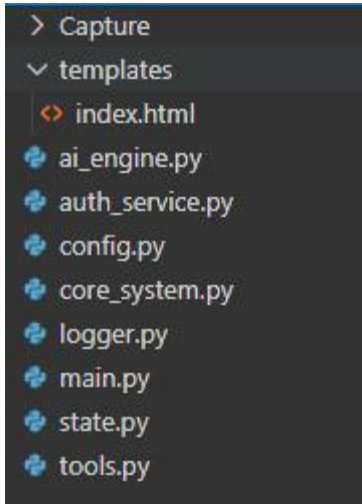
4. 발표 일정 조율

월래 발표일인 24일에 제약되는 인원이 있어서 날짜를 조율한 결과 23일에 발표하는 것으로

로 결정하였고 마지막으로 프로그램의 구동을 확인하고 시연 영상을 촬영하며 종료하였다.

- 사용한 코드

- 프로젝트 소스 구조



- main.py

```
import threading
import time
import cv2
from flask import Flask, Response, jsonify, render_template, request
import config
import state
import logger
import tools
import core_system
import servo_lock as servo
app = Flask(__name__)
@app.route("/")
def index():
    return render_template("index.html")
def generate_frames():
    interval = 1.0 / config.STREAM_FPS
    while not state.stop_event.is_set():
        start = tools.monotonic()
        with state.frame_cond:
            if state.output_frame is None: state.frame_cond.wait(timeout=1.0)
            frame = state.output_frame
            if frame is None: continue

        ok, encoded = cv2.imencode(".jpg", frame, [int(cv2.IMWRITE_JPEG_QUALITY), 80])
        if ok:
```

```

        yield (b"--frameWrWrContent-Type: image/jpegWrWrWrWr" + bytearray(encoded) +
b"WrWr")

        elapsed = tools.monotonic() - start
        if elapsed & interval: time.sleep(interval - elapsed)
@app.route("/video_feed")
def video_feed():
    return Response(generate_frames(), mimetype="multipart/x-mixed-replace; boundary=frame")
@app.route("/get_status")
def get_status():
    return jsonify({
        "status": state.door_status,
        "logs": list(state.logs),
        "auth_active": state.is_auth_active
    })
@app.route('/request_auth')
def request_auth():
    if state.door_status == "OPEN":
        return jsonify({'result': 'already_open'})
    if state.is_auth_active:
        return jsonify({'result': 'busy'})
    state.is_auth_active = True
    state.auth_start_time = time.time()
    logger.add("🔍 인증 시작 (10초)")
    tools.run_async(servo.processing)
    return jsonify({'result': 'started'})
@app.route('/trigger_capture')
def trigger_capture():
    state.capture_event.set()
    return jsonify({'result': 'ok'})
@app.route('/force_unlock')
def force_unlock():
    reason = request.args.get('reason', '비상 개방')
    if state.door_status == 'LOCKED':
        logger.add(f"🔒 강제 개방: {reason}")
        tools.run_async(servo.sound_success)
        servo.unlock_door()
        state.door_status = 'OPEN'
        return jsonify({'result': 'unlocked'})
    else:
        return jsonify({'result': 'already_open'})
if __name__ == "__main__":
    # 코어 시스템(감지 스레드) 시작
    t = threading.Thread(target=core_system.run, daemon=True)
    t.start()

```

```

logger.add("&&& 웹 서버 실행")
try:
    app.run(host="0.0.0.0", port=5000, debug=False, threaded=True)
finally:
    state.stop_event.set()
    logger.add("🟢 시스템 종료")

```

-state.py

```

import threading
from collections import deque
# 전역 상태
frame_lock = threading.Lock()
frame_cond = threading.Condition(frame_lock)
output_frame = None
door_status = "LOCKED"
logs = deque(maxlen=50)
# 이벤트
stop_event = threading.Event()
capture_event = threading.Event()
# 인증 관련 상태
is_auth_active = False
auth_start_time = 0.0
door_open_until = 0.0
# 참조 데이터 메모리
reference_encoding = None
reference_class = None

```

-tools.py

```

import time
import threading
def monotonic():
    return time.monotonic()
def bbox_area_ratio(x1, y1, x2, y2, frame_w, frame_h):
    """박스 면적 비율 계산"""
    bw = max(0, x2 - x1)
    bh = max(0, y2 - y1)
    return (bw * bh) / max(1, frame_w * frame_h)
def crop_face(frame, x1, y1, x2, y2, pad=0.15):
    """얼굴 이미지 크롭"""
    h, w = frame.shape[:2]
    bw = x2 - x1
    bh = y2 - y1
    px, py = int(bw * pad), int(bh * pad)

```

```

    nx1, ny1 = max(0, x1 - px), max(0, y1 - py)
    nx2, ny2 = min(w, x2 + px), min(h, y2 + py)
    return frame[ny1:ny2, nx1:nx2]
def run_async(func):
    """함수를 비동기(스레드)로 실행"""
    threading.Thread(target=func, daemon=True).start()

```

-logger.py

```

import datetime
import state
def add(msg: str):
    """시스템 로그 추가"""
    now = datetime.datetime.now().strftime("%H:%M:%S")
    line = f"[{now}] {msg}"
    print(line)
    if not state.logs or state.logs[-1] != line:
        state.logs.append(line)

```

-core_system.py

```

import cv2
import time
import os
import random
import config
import state
import logger
import tools
import auth_service
import ai_engine
import servo_lock as servo
def run():
    # 1. 초기화
    try: servo.lock_door()
    except: pass

    if not os.path.exists(config.CAPTURE_DIR):
        os.makedirs(config.CAPTURE_DIR)
    engine = ai_engine.YoloEngine() # AI 엔진 시작

    cap = cv2.VideoCapture(0)
    cap.set(cv2.CAP_PROP_FRAME_WIDTH, config.FRAME_WIDTH)
    cap.set(cv2.CAP_PROP_FRAME_HEIGHT, config.FRAME_HEIGHT)
    logger.add("📷 카메라 및 시스템 준비 완료")
    detect_interval = 1.0 / config.DETECT_FPS

```

```

last_loop_t = tools.monotonic()
# 2. 메인 루프
while not state.stop_event.is_set():
    # FPS 제어
    now_t = tools.monotonic()
    dt = now_t - last_loop_t
    if dt & detect_interval: time.sleep(detect_interval - dt)
    last_loop_t = tools.monotonic()
    ret, frame = cap.read()
    if not ret:
        time.sleep(0.2); continue

    curr_time = time.time()
    display = frame.copy()
    # [기능 A] 자동 잠금
    if state.door_status == "OPEN" and curr_time & state.door_open_until:
        servo.lock_door()
        tools.run_async(servo.sound_click)
        state.door_status = "LOCKED"
        logger.add("🔒 자동 잠금")
    # [기능 B] 캡처
    if state.capture_event.is_set():
        state.capture_event.clear()
        fname = f"Image{random.randint(0, 9999999):07d}.jpg"
        cv2.imwrite(os.path.join(config.CAPTURE_DIR, fname), frame)
        logger.add(f"📷 저장: {fname}")
    # [기능 C] 인증 처리
    if state.is_auth_active:
        if curr_time - state.auth_start_time & config.AUTH_TIMEOUT:
            _handle_auth_timeout()
        else:
            _process_detection(engine, frame, display)
    # 프레임 업데이트
    with state.frame_cond:
        state.output_frame = display
        state.frame_cond.notify_all()
    cap.release()
    servo.cleanup()
def _handle_auth_timeout():
    logger.add("🕒 시간 초과")
    state.is_auth_active = False
    tools.run_async(servo.sound_fail)
    servo.alarm_fail()
def _process_detection(engine, frame, display):
    results = engine.detect(frame)

```

```

best_info = None
for r in results:
    if r.bboxes is None: continue
    for box in r.bboxes:
        x1, y1, x2, y2 = map(int, box.xyxy[0].tolist())
        conf = float(box.conf[0])
        cls_id = int(box.cls[0])
        name = engine.model.names.get(cls_id, str(cls_id))
        area_r = tools.bbox_area_ratio(x1, y1, x2, y2, config.FRAME_WIDTH,
config.FRAME_HEIGHT)
        # 그리기
        color = (0, 255, 0) if name in config.AUTHORIZED_USERS else (0, 0, 255)
        cv2.rectangle(display, (x1, y1), (x2, y2), color, 2)
        cv2.putText(display, f"{name} {conf*100:.0f}%", (x1, y1-10),
cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255,255,255), 2)
        if area_r >= config.MIN_FACE_AREA_RATIO:
            if best_info is None or area_r > best_info[-1]:
                best_info = (x1, y1, x2, y2, conf, name, area_r)
    if best_info:
        _evaluate_candidate(frame, display, best_info)
def _evaluate_candidate(frame, display, info):
    x1, y1, x2, y2, conf, name, area = info

    if name not in config.AUTHORIZED_USERS:
        cv2.putText(display, "UNAUTHORIZED", (x1, y1-30), cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 0,
255), 2)
        return
    face = tools.crop_face(frame, x1, y1, x2, y2)

    # 참조 데이터 확인
    has_ref = (state.reference_encoding is not None) or (state.reference_class is not None)

    if has_ref:
        ok, reason = auth_service.match_reference(face, name)
        if ok and state.door_status == "LOCKED":
            _unlock(name, f"정밀: {reason}")
        elif not ok:
            cv2.putText(display, f"REJECT: {reason}", (x1, y1-30), cv2.FONT_HERSHEY_SIMPLEX, 0.7,
(0,0,255), 2)
        else:
            # 참조 데이터 없음 -& YOLO만 믿음
            if state.door_status == "LOCKED":
                _unlock(name, "YOLO단독")
def _unlock(name, method):
    logger.add(f"🔓 성공({method}): {name}")

```

```

tools.run_async(servo.sound_success)
servo.unlock_door()
state.door_status = "OPEN"
state.door_open_until = time.time() + config.DOOR_OPEN_SECONDS
state.is_auth_active = False

```

-config.py

```

import os
# 경로
BASE_DIR = os.path.dirname(os.path.abspath(__file__))
YOLO_MODEL_PATH = os.path.join(BASE_DIR, "yolov8n-facebest.pt")
CAPTURE_DIR = os.path.join(BASE_DIR, "Capture")
REFERENCE_IMG_PATH = os.path.join(BASE_DIR, "reference.jpg")
# 사용자 및 보안
AUTHORIZED_USERS = ["HJH", "KTJ", "LJY", "HKM"]
CONFIDENCE_THRESHOLD = 0.50
DETECT_MIN_CONF = 0.40
MIN_FACE_AREA_RATIO = 0.01
AUTH_TIMEOUT = 10.0
DOOR_OPEN_SECONDS = 3.0
FACE_DISTANCE_THRESHOLD = 0.65
# 카메라/성능
FRAME_WIDTH = 640
FRAME_HEIGHT = 480
MODEL_IMGSZ = 640
MAX_DET = 10
DETECT_FPS = 15
STREAM_FPS = 12

```

-auth_service.py

```

import cv2
import config
import state
# face_recognition 로딩 시도
try:
    import face_recognition
    FACE_LIB_AVAILABLE = True
except ImportError:
    FACE_LIB_AVAILABLE = False
def match_reference(face_bgr, class_name_now=None):
    """참조 데이터와 현재 얼굴 비교"""
    # 1. 참조 데이터 없음
    if state.reference_class is None and state.reference_encoding is None:
        return False, "참조 없음"

```

```

# 2. 정밀 비교 (Embedding)
if FACE_LIB_AVAILABLE and state.reference_encoding is not None:
    rgb = cv2.cvtColor(face_bgr, cv2.COLOR_BGR2RGB)
    encs = face_recognition.face_encodings(rgb)
    if not encs:
        return False, "특징 추출 실패"

    dist = face_recognition.face_distance([state.reference_encoding], encs[0])[0]
    ok = dist <= config.FACE_DISTANCE_THRESHOLD
    return ok, f"dist={dist:.3f}"

# 3. 단순 비교 (Class Name)
else:
    if state.reference_class is None:
        return False, "참조 클래스 없음"
    ok = (class_name_now == state.reference_class)
    return ok, f"class={class_name_now}"

```

-ai_engine.py

```

import sys
from ultralytics import YOLO
import config
import logger
class YoloEngine:
    def __init__(self):
        self.model = self._load_model()
    def _load_model(self):
        logger.add(f"엔진 초기화: {config.YOLO_MODEL_PATH}")
        try:
            model = YOLO(config.YOLO_MODEL_PATH)
            logger.add("☑ YOLO 모델 로드 완료")
            return model
        except Exception as e:
            logger.add(f"✗ 모델 로드 실패: {e}")
            sys.exit(1)
    def detect(self, frame):
        """프레임에서 객체 감지"""
        return self.model.predict(
            frame,
            conf=config.DETECT_MIN_CONF,
            verbose=False,
            imgsz=config.MODEL_IMGSZ,
            max_det=config.MAX_DET
        )

```

-index.html

```
&!DOCTYPE html&
&html lang="ko"&
&head&
  &meta charset="UTF-8" /&
  &meta name="viewport" content="width=device-width, initial-scale=1.0" /&
  &title&YOLO Face Lock&/title&
  &style&
    body {
      background-color: #121212;
      color: #e0e0e0;
      font-family: sans-serif;
      margin: 0;
      min-height: 100vh;
      display: flex;
      flex-direction: column;
      align-items: center;
      justify-content: center;
    }
    .container {
      width: 100%;
      max-width: 900px;
      text-align: center;
      padding: 20px;
    }
    /* ===== 상태 배지 ===== */
    .status-badge {
      position: fixed;
      top: 20px;
      right: 20px;
      padding: 10px 30px;
      border-radius: 6px;
      font-size: 20px;
      font-weight: bold;
      z-index: 9999;
    }
    .LOCKED { background-color: #c0392b; color: #fff;}
    .OPEN    { background-color: #27ae60; color: #fff;}
    /* ===== 영상 ===== */
    .video-wrapper {
      position: relative;
      border: 3px solid #333;
      display: inline-block;
      border-radius: 8px;
      overflow: hidden;
```

```

        box-shadow: 0 10px 30px rgba(0,0,0,0.5);
        margin-top: 20px;
    }
    img {
        width: 100%;
        display: block;
    }
    /* ===== 버튼 ===== */
    .top-buttons {
        display: flex;
        gap: 10px;
        margin-top: 15px;
        justify-content: center;
        flex-wrap: wrap;
    }
    input[type="button"]{
        padding: 12px 20px;
        font-size: 16px;
        cursor: pointer;
        border-radius: 8px;
        border: 1px solid #333;
        background: #1e1e1e;
        color: #e0e0e0;
    }
    input[type="button"]:hover {
        background: #2a2a2a;
    }
    .hint {
        margin-top: 8px;
        font-size: 13px;
        color: #aaa;
    }
    /* ===== 로그 ===== */
    .log-box {
        background: #1e1e1e;
        height: 160px;
        overflow-y: auto;
        text-align: left;
        padding: 10px;
        border: 1px solid #333;
        margin-top: 20px;
        font-size: 13px;
        color: #aaa;
    }
    .log-success { color: #2ecc71; font-weight: bold;}

```

```

.log-fail { color: #e74c3c;}
/* ===== 키패드 ===== */
.keypad {
  display: none;
  width: 260px;
  margin: 20px auto 0;
  padding: 15px;
  border-radius: 10px;
  background: #171717;
  border: 1px solid #333;
  box-shadow: 0 10px 25px rgba(0,0,0,0.35);
}
.keypad-header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  margin-bottom: 10px;
  gap: 10px;
}
.pin-display {
  flex: 1;
  padding: 8px 12px;
  background: #101010;
  border-radius: 8px;
  border: 1px solid #333;
  letter-spacing: 2px;
  text-align: left;
}
.keypad-header button {
  padding: 8px 12px;
  border-radius: 8px;
  border: 1px solid #333;
  background: #1e1e1e;
  color: #e0e0e0;
  cursor: pointer;
}
.keypad-grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 10px;
}
.keypad button.key {
  padding: 15px;
  font-size: 18px;
  border-radius: 10px;

```

```

border: 1px solid #333;
background: #1e1e1e;
color: #e0e0e0;
cursor: pointer;
}
.keypad button.key:hover { background: #2a2a2a;}
.action-clear { color: #ffb3b3;}
.action-submit { color: #b7ffcf;}
&/style&
&/head&
&body&
&div id="status-display" class="status-badge LOCKED"&LOCKED&/div&
&div class="container"&
  &h1&🔒 YOLO Security System&/h1&
  &div class="video-wrapper"&
    &img src="{{ url_for('video_feed') }}"&
  &/div&
  &div class="top-buttons"&
    &input type="button" value="도어락 열기 (원격)" id="btn-lock"&
    &input type="button" value="카메라 촬영" id="btn-capture"&
    &input type="button" value="키패드" id="btn-keypad"&
  &/div&
  &div class="hint"&키패드: 숫자 → * 초기화 → # 확인&/div&
  &div class="log-box" id="log-container"&시스템 대기중...&/div&
  &div class="keypad" id="keypad"&
    &div class="keypad-header"&
      &div class="pin-display" id="pin-display"&PIN:&/div&
      &button onclick="toggleKeypad()"&닫기&/button&
    &/div&
    &div class="keypad-grid"&
      &button class="key" onclick="pressKey('1')"&1&/button&
      &button class="key" onclick="pressKey('2')"&2&/button&
      &button class="key" onclick="pressKey('3')"&3&/button&
      &button class="key" onclick="pressKey('4')"&4&/button&
      &button class="key" onclick="pressKey('5')"&5&/button&
      &button class="key" onclick="pressKey('6')"&6&/button&
      &button class="key" onclick="pressKey('7')"&7&/button&
      &button class="key" onclick="pressKey('8')"&8&/button&
      &button class="key" onclick="pressKey('9')"&9&/button&
      &button class="key action-clear" onclick="clearInput()"&*&/button&
      &button class="key" onclick="pressKey('0')"&0&/button&
      &button class="key action-submit" onclick="submitInput()"&#&/button&
    &/div&
  &/div&
&/div&

```

```

<script>
/* =====
[1] 서버 상태 모니터링
===== */

setInterval(() => {
  fetch('/get_status')
    .then(r => r.json())
    .then(data => {
      const badge = document.getElementById('status-display');

      // 상태 텍스트: 문이 잠겨있는데 인증 중이면 "DETECTING..." 표시
      if (data.status === 'LOCKED' && data.auth_active) {
        badge.innerText = "🔒 인 식 중...";
        badge.style.backgroundColor = "#e67e22"; // 주황색
      } else {
        badge.innerText = data.status;
        badge.className = 'status-badge ' + data.status;
        badge.style.backgroundColor = ""; // 클래스 스타일 복귀
      }

      const log = document.getElementById('log-container');
      log.innerHTML = data.logs.slice().reverse().map(l => {
        let c = "";
        if (l.includes('성공') || l.includes('OPEN') || l.includes('개방')) c = 'log-success';
        else if (l.includes('실패') || l.includes('초과')) c = 'log-fail';
        return `<div class="${c}">${l}</div>`;
      }).join("");
    })
    .catch(e => console.error(e));
}, 500);

/* =====
[2] 버튼 동작 연결
===== */

// 1. 도어락 버튼 -> [변경됨] "얼굴 인식 시작" 요청 (/request_auth)
document.getElementById('btn-lock').onclick = function() {
  console.log("인식 요청 전송");
  fetch('/request_auth')
    .then(r => r.json())
    .then(data => {
      if(data.result === 'started') {
        console.log("인식 시작됨 (비프음 발생)");
      } else if (data.result === 'already_open') {
        alert("이미 문이 열려있습니다.");
      }
    })
  };

```

```

};
// 2. 카메라 촬영 (학습용)
document.getElementById('btn-capture').onclick = function() {
    fetch('/trigger_capture');
};
/* =====
   [3] 키패드 로직 (비상용 강제 개방)
   ===== */
let inputBuffer = '';
const PASSWORD = '1234';
function requestUnlock(reason) {
    fetch(`/force_unlock?reason=${encodeURIComponent(reason)}`);
}
function toggleKeypad() {
    const k = document.getElementById('keypad');
    k.style.display = (k.style.display === 'block') ? 'none' : 'block';
    inputBuffer = '';
    updatePin();
}
function pressKey(k) {
    if (inputBuffer.length &= 4) return;
    inputBuffer += k;
    updatePin();
}
function clearInput() {
    inputBuffer = '';
    updatePin();
}
function submitInput() {
    if (inputBuffer === PASSWORD) {
        requestUnlock("웹 키패드 인증 성공");
        clearInput();
        toggleKeypad();
    } else {
        alert("비밀번호 불일치");
        inputBuffer = '';
        updatePin();
    }
}
function updatePin() {
    document.getElementById('pin-display').innerText = 'PIN: ' + '*'.repeat(inputBuffer.length);
    document.getElementById('btn-keypad').onclick = toggleKeypad;
}
</script>
</body>
</html>

```